

Module-2

1. Fraud Transaction Detection (Linear Search)

* Problem Statement -

A bank stores transaction IDs in the order they occur. Since transactions are not sorted, check whether a suspicious transaction ID exists.

* Approach -

- Input ~~n~~ transaction IDs in array format.
- Input target transaction ID inside the array.
- Traverse ~~from~~ array from first to last element.
- ~~If~~ From Traversing, target is found, so print "Fraud Transaction Found", otherwise print "Transaction Not Found".

* Code -

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n; cout << "Enter the size of array: ";
    cin >> n;

    int arr[n];
    cout << "Enter the elements of an array: ";
    for(int i=0; i<n; i++) {
        cin >> arr[i];
    }

    int target;
    cout << "Enter the target value: ";
    cin >> target;

    bool found = false;
```



```

for (int i=0; i<n; i++){
    if (arr[i] == target){
        found = true;
        break;
    }
}

if (found){
    cout << "Fraud Transaction Found";
} else {
    cout << "Transaction Not Found";
}

return 0;
}

```

* Time Complexity:- $O(n)$

2. Server Log Search (Binary Search)

* Problem Statement:-

A cloud company stores server log IDs in sorted order. Find whether a target log ID exists.

* Approach -

- Input n ~~server~~ server log IDs in array format with sorted order.
- Input target server log ID in inside the array.
- Initialize $low = 0$ & $high = n-1$.
- Find middle element.
- If middle element equal to target, then stop.
- If target is smaller, search in left half.
- Otherwise search in right half.
- Repeat until target not found or $low > high$.

Code -

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter the size of array: ";
    cin >> n;

    int arr[n];
    cout << "Enter the element of array: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    int target;
    cout << "Enter the value of target: ";
    cin >> target;

    int st = 0, end = n - 1;
    bool found = false;

    while (st <= end) {
        int mid = st + (end - st) / 2;
        if (arr[mid] == target) {
            found = true;
            break;
        }
        else if (target < arr[mid]) {
            high = mid - 1;
        }
        else {
            low = mid + 1;
        }
    }

    if (found) {
        cout << "Log found";
    }
    else {
        cout << "Log not found";
    }
    return 0;
}
```


3. Sort Employee Salaries (Bubble Sort)

Problem Statement -

Sort employee salaries in ascending order.

Approach -

- Compare adjacent elements.
- Swap if left element is greater.
- Repeat for all passes.
- Largest element reaches the end after every pass.

Code -

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int arr[n];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }

    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```


4. Sort Website Response Time (Insertion sort)

* Problem Statement

Sort website response times in ascending order.

* Approach

- Assume first element is sorted.
- Pick next element as Key.
- Shift greater elements to the right.
- Insert Key at correct position.
- Repeat until array becomes sorted.

* Code -

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int arr[n];
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}
```


5. Prioritize Bug Reports (Selection Sort)

Problem Statement -

Sort bug priority score in ascending order.

Approach -

- Find minimum element.
- Swap it with current position.
- Repeat for remaining array.
- Continue until array becomes sorted.

Code -

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int arr[n];
    for(int i=0; i<n; i++)
        cin >> arr[i];

    for(int i=0; i<n-1; i++){
        int minIndex = i;
        for(int j=i+1; j<n; j++){
            if(arr[j] < arr[minIndex])
                minIndex = j;
        }
        swap(arr[i], arr[minIndex]);
    }

    for(int i=0; i<n; i++){
        cout << arr[i] << " ";
    }

    return 0;
}
```

1. Fraud Transaction Detection (Linear Search)

Problem Statement

A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the security team needs to check if a suspicious transaction ID exists.

Write a program using Linear Search.

Input Format

- First line contains integer N
- Second line contains N transaction IDs
- Third line contains suspicious transaction ID

Output Format

Print: Fraud Transaction Found or Transaction Not Found

```
ps1.cpp > main()
1  #include<iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      cout << "Enter the number of transactions: ";
7      cin >> n;
8
9      int arr[n];
10     cout << "Enter the transaction amounts: ";
11     for(int i=0;i<n;i++)
12         cin >> arr[i];
13
14     int target;
15     cout << "Enter the target transaction amount: ";
16     cin >> target;
17
18     bool found = false;
19
20     for(int i=0;i<n;i++) {
21         if(arr[i] == target) {
22             found = true;
23             break;
24         }
25     }
26
27     if(found)
28         cout << "Fraud Transaction Found";
29     else
30         cout << "Transaction Not Found";
31
32     return 0;
33 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps1.cpp
PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
Enter the number of transactions: 5
Enter the transaction amounts: 1201
1305
1450
140
1502
Enter the target transaction amount: 1450
Fraud Transaction Found
```

2. Server Log Search (Binary Search)

Problem Statement

A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID exists. Write a program using Binary Search.

Input Format

- First line contains N
- Second line contains sorted log IDs
- Third line contains target log ID

Output Format

Print: Log Found or Log Not Found

Constraints

- $1 \leq N \leq 10^5$


```
C++ ps2.cpp > main()
1  #include<iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      cout << "Enter the number of logs: ";
7      cin >> n;
8
9      int arr[n];
10     cout << "Enter the log elements: ";
11     for(int i=0;i<n;i++)
12         cin >> arr[i];
13
14     int target;
15     cout << "Enter the target element: ";
16     cin >> target;
17
18     int low = 0, high = n-1;
19     bool found = false;
20
21     while(low <= high) {
22         int mid = (low + high) / 2;
23
24         if(arr[mid] == target) {
25             found = true;
26             break;
27         }
28         else if(target < arr[mid])
29             high = mid - 1;
30         else
31             low = mid + 1;
32     }
33
34     if(found)
35         cout << "Log Found";
36     else
37         cout << "Log Not Found";
38
39     return 0;
40 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps2.cpp
PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
Enter the number of logs: 7
Enter the log elements: 1001
1005
1010
1015
1020
1030
1041
Enter the target element: 1015
Log Found
```

3. Sort Employee Salaries (Bubble Sort)

Problem Statement

A company wants to sort employee salaries in ascending order for payroll analysis. Write a program using Bubble Sort.

Input Format

- First line contains N
- Second line contains salaries

Output Format

Print sorted salaries.

```
C++ ps3.cpp > main()
1  #include<iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      cout << "Enter the number of elements: ";
7      cin >> n;
8
9      int arr[n];
10     cout << "Enter the elements: ";
11     for(int i=0;i<n;i++)
12         cin >> arr[i];
13
14     for(int i=0;i<n-1;i++) {
15         for(int j=0;j<n-i-1;j++) {
16             if(arr[j] > arr[j+1]) {
17                 swap(arr[j], arr[j+1]);
18             }
19         }
20     }
21
22     cout << "Sorted elements: ";
23     for(int i=0;i<n;i++)
24         cout << arr[i] << " ";
25
26     return 0;
27 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps3.cpp
PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
Enter the number of elements: 5
Enter the elements: 4500 3200 5500 2800 6000
Sorted elements: 2800 3200 4500 5500 6000
```

4. Sort Website Response Time (Insertion Sort)

Problem Statement

A web monitoring system records website response times. New data comes continuously and must be inserted into the correct position. Write a program using Insertion Sort.

Input Format

- First line contains N
- Second line contains response times

Output Format

Print sorted response times.

```
C++ ps4.cpp > main()
1  #include<iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      cout << "Enter the number of elements: ";
7      cin >> n;
8
9      int arr[n];
10     cout << "Enter the elements: ";
11     for(int i=0;i<n;i++)
12         cin >> arr[i];
13
14     for(int i=1;i<n;i++) {
15         int key = arr[i];
16         int j = i-1;
17
18         while(j>=0 && arr[j]>key) {
19             arr[j+1] = arr[j];
20             j--;
21         }
22
23         arr[j+1] = key;
24     }
25
26     cout << "Sorted elements: ";
27     for(int i=0;i<n;i++)
28         cout << arr[i] << " ";
29
30     return 0;
31 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps4.cpp
● PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
Enter the number of elements: 5
Enter the elements: 250 120 320 170 100
Sorted elements: 100 120 170 250 320
```

5. Prioritize Bug Reports (Selection Sort)

Problem Statement

A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to sort priorities in ascending order. Write a program using Selection Sort.

Input Format

- First line contains N
- Second line contains priority scores

Output Format

Print sorted priority scores.


```
ps5.cpp > main()
5   int n;
6   cout << "Enter the number of elements: ";
7   cin >> n;
8
9   int arr[n];
10  cout << "Enter the elements: ";
11  for(int i=0;i<n;i++)
12      cin >> arr[i];
13
14  for(int i=0;i<n-1;i++) {
15
16      int minIndex = i;
17
18      for(int j=i+1;j<n;j++) {
19          if(arr[j] < arr[minIndex])
20              minIndex = j;
21      }
22
23      swap(arr[i], arr[minIndex]);
24  }
25  cout << "Sorted elements: ";
26  for(int i=0;i<n;i++)
27      cout << arr[i] << " ";
28
29  return 0;
30 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\manoj\OneDrive\Documents\DSA C++> g++ ps5.cpp
PS C:\Users\manoj\OneDrive\Documents\DSA C++> .\a.exe
Enter the number of elements: 5
Enter the elements: 9 2 7 1 5
Sorted elements: 1 2 5 7 9
```